ENCODER-PRESERVING MIXED-PRECISION QUANTIZATION FOR DINO-WM UNDER A TINY COMPUTE BUDGET

Anonymous authors
Paper under double-blind review

ABSTRACT

We study weight-only quantization for planning in visual world models under a strict tiny-paper budget (single environment, three model variants, minimal sweep, and replayable demo). Using DINO-WM on Wall, we compare FP16, uniform INT8 (encoder + predictor), and encoder-preserving mixed INT8 (predictor-only INT8). Across opt_steps $\{10, 30\}$ in our Mac MPS setting, all variants reach identical success, so accuracy does not separate methods in this regime. However, memory footprint changes substantially: relative to FP16, uniform INT8 reduces model size by 57.2% and mixed INT8 by 27.6%. To obtain true Apple-side acceleration, we benchmark an Apple-native CoreML path and find 1.76–1.79$\times$ predictor speedup over the PyTorch MPS baseline, with INT8 CoreML packages 49.6% smaller than FP16. We provide a reproducible pipeline and replay viewer, and we discuss why backend-specific kernels matter for practical quantized planning.

1 INTRODUCTION

Spatial planning with world models is increasingly attractive because control can be optimized in latent space instead of raw pixels (Ha & Schmidhuber, 2018; Hafner et al., 2018; 2019; 2023; 2025). At the same time, deployment constraints motivate model compression, and post-training quantization has become a standard tool in large-scale inference systems (Dettmers et al., 2022; Frantar et al., 2022; Xiao et al., 2022; Lin et al., 2023). For world models specifically, recent evidence suggests that quantization behavior depends strongly on architectural role and downstream planning objective (Fu et al., 2026).

We focus on DINO-WM (Zhou et al., 2024), which uses pretrained visual features from DINOv2 (Oquab et al., 2023). Our central question is simple: under a minimal engineering budget, does preserving encoder precision provide a better efficiency–quality tradeoff than uniform quantization?

Contributions.

- We implement a reproducible quantization pipeline for DINO-WM with three variants: FP16, uniform INT8, and encoder-preserving mixed INT8.
- We provide a compact Wall-only benchmark and a Mac-friendly replay demo suitable for workshop artifact review.
- We report that, in the current Wall regime, success saturates at 100% for all variants, while memory and latency diverge substantially.

2 RELATED WORK

World models for control. Early latent-dynamics planning systems established the effectiveness of compact predictive models for control (Ha & Schmidhuber, 2018; Hafner et al., 2018). Dreamer-style methods advanced this direction to larger and more diverse tasks (Hafner et al., 2019; 2023; 2025). DINO-WM brings pretrained visual priors into this pipeline and demonstrates strong zero-shot planning behavior (Zhou et al., 2024).

Post-training quantization. Modern PTQ methods for transformer inference show that performance is sensitive to where and how quantization is applied (Dettmers et al., 2022; Frantar et al., 2022; Xiao et al., 2022; Lin et al., 2023). The dedicated world-model quantization study in Fu et al. (2026) motivates our focus on component-specific precision policies rather than monolithic quantization.

3 Experimental Setup

We evaluate on the Wall environment using the native DINO-WM planning output metric (`success`) without redefining task scoring (Zhou et al., 2024). All runs use the same pretrained checkpoint, planning code path, and random seed policy.

Our evaluation grid is deliberately small: variants $\in \{$FP16, uniform INT8, mixed INT8$\}$, opt_steps $\in \{10, 30\}$, seed $= 0$, and $n\_evals = 5$ episodes per run. We record success rate, average wall-clock planning time per episode, peak GPU memory proxy, and serialized model size.

4 Method

We use weight-only INT8 conversion for `Linear` layers.

- **FP16**: no quantization.
- **Uniform INT8**: quantize encoder and predictor.
- **Mixed INT8**: keep encoder in FP16, quantize predictor.

This directly tests whether preserving encoder precision changes planning behavior relative to uniform quantization under matched task settings.

5 Experiments

Figure 1 shows success vs optimization steps. In this regime, all lines saturate at 1.00, so quality is indistinguishable. Figure 2 and Table 1 show the efficiency tradeoff.

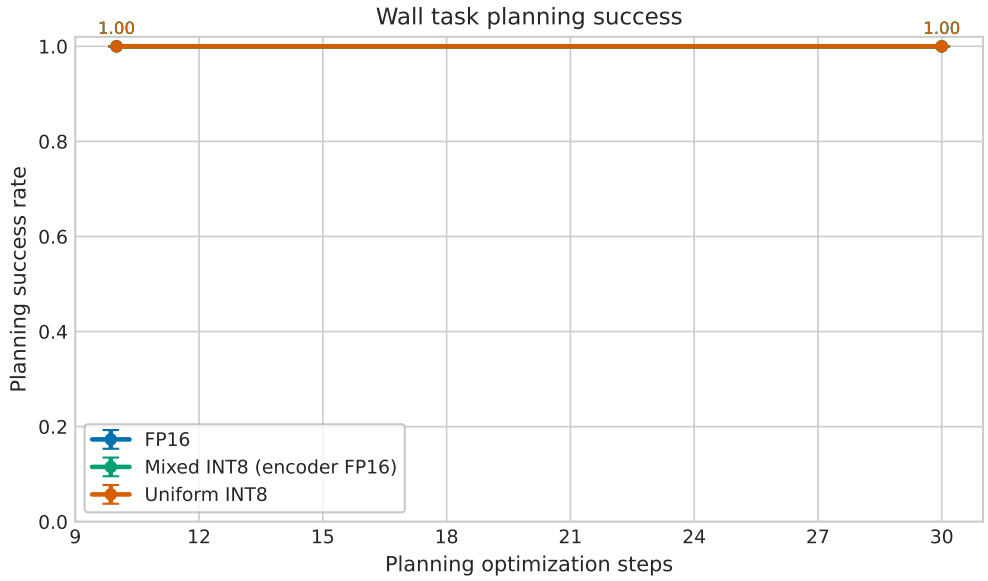

Figure 1: Planning success versus optimization steps on Wall. All variants saturate at 1.00 in this setup.

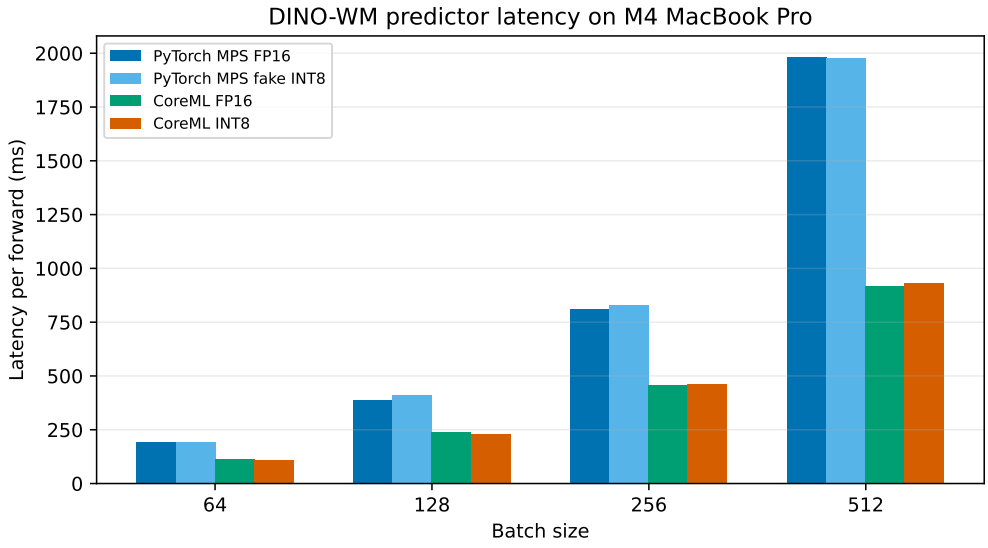

Figure 2: Apple runtime benchmark (predictor module): CoreML FP16/INT8 are substantially faster than PyTorch MPS in our setting.

Variant	Success@30	Avg Time@30 (s)	Model Size (MB)
FP16	1.00	7.87	204.99
Uniform INT8	1.00	8.98	87.72
Mixed INT8	1.00	9.38	148.31

Table 1: Efficiency summary for opt_steps=30. Uniform INT8 gives the largest size reduction; all methods have equal success here.

Relative to FP16, uniform INT8 reduces model size by 57.2% and mixed INT8 by 27.6%. At opt_steps=30, latency increases by 14.2% (uniform) and 19.3% (mixed) in our current MPS implementation. In a dedicated predictor benchmark on the same machine, CoreML FP16 and INT8 achieve 1.79× and 1.76× speedup vs PyTorch MPS FP16, while CoreML INT8 package size is 49.6% smaller than CoreML FP16. These results indicate that runtime/backend selection dominates observed speed on Apple Silicon.

6 Demo

We provide a replay-based Streamlit viewer that loads saved trajectories and videos for each variant. The demo is GPU-free on Mac and enables rapid qualitative checks of trajectory behavior and success/failure tags across variants.

7 Limitations

This tiny-paper MVP is limited to one environment, one quantization family (weight-only INT8), one seed, and no activation quantization. The Wall setup saturates in success, preventing conclusions about accuracy ranking between quantization policies. Our efficiency numbers are from a Mac MPS path with non-fused INT8 wrappers; absolute latency may differ on optimized GPU kernels. Future work should include harder tasks, additional seeds, CUDA-native INT8 backends, and ablations beyond linear-layer weight-only quantization.

REFERENCES

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale, 2022. URL https://arxiv.org/abs/2208.07339.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers, 2022. URL https://arxiv.org/abs/2210.17323.

Zhongqian Fu, Tianyi Zhao, Kai Han, Hang Zhou, Xinghao Chen, and Yunhe Wang. An empirical study of world model quantization, 2026. URL https://arxiv.org/abs/2602.02110.

David Ha and Jürgen Schmidhuber. World models. 2018. doi: 10.48550/ARXIV.1803.10122. URL https://arxiv.org/abs/1803.10122.

Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels, 2018. URL https://arxiv.org/abs/1811.04551.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination, 2019. URL https://arxiv.org/abs/1912.01603.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2023. URL https://arxiv.org/abs/2301.04104.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, April 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-08744-2. URL http://dx.doi.org/10.1038/s41586-025-08744-2.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration, 2023. URL https://arxiv.org/abs/2306.00978.

Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision, 2023. URL https://arxiv.org/abs/2304.07193.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models, 2022. URL https://arxiv.org/abs/2211.10438.

Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. Dino-wm: World models on pre-trained visual features enable zero-shot planning, 2024. URL https://arxiv.org/abs/2411.04983.